GRAIN-CAN: Detecting Cross-Vehicle CAN Intrusions with Same-ID Residuals*

Qi'ao Li[1], Simiao Gao[2], Pengcheng Wang[1**], and Yuran Li[3]

[1] School of Cyber Science and Technology, Beihang University, Beijing, China
{qi_aolee,pcwang}@buaa.edu.cn
[2] School of Transportation Science and Engineering, Beihang University, Beijing, China
GoesM@buaa.edu.cn
[3] Automotive Standardization Research Institute, China Automotive Technology and Research Center Co., Ltd., Tianjin, China
liyuran@catarc.ac.cn

Abstract. Modern CAN intrusion detection must identify rare attacks after both the vehicle and attack pattern shift. Raw fixed-window detectors are brittle in this setting: a window may contain many benign frames and only a few abnormal messages, causing local timing or payload evidence to be diluted. This paper presents GRAIN-CAN, a lightweight supervised IDS pipeline that exposes same-ID residuals before aggregation. GRAIN-CAN maintains a per-ID history state and converts each incoming frame into history-only timing-gap, payload-change, payload-statistical, and local ID-behavior residuals; fixed-window summaries are then classified by a tree-based detector. We evaluate GRAIN-CAN with attack-positive precision, recall, attack-F1, false-negative rate, false-positive rate, and confusion matrices, and report AUPR and Recall@FPR as supplementary score-based operating metrics. In the hardest joint vehicle-and-attack shift, GRAIN-CAN increases thresholded attack recall by 4.5× over the raw-window control and reaches 80.5% Recall@FPR=10^{-3}, reducing the corresponding false-negative rate to 19.5%. CPU-only inference processes about 97k frames/s.

1 Introduction

Modern vehicles rely on Controller Area Network (CAN) buses to connect electronic control units. The classical CAN protocol was designed for reliability and low-latency broadcast communication rather than message authentication. This design has enabled practical attacks through malicious message injection, spoofing, and remote attack surfaces [1–3]. CAN intrusion detection systems

* This work is supported by the National Natural Science Foundation of China (62573030), Zhejiang Provincial Natural Science Foundation of China (LMS26E080008), Beijing-Tianjin-Hebei Basic Research Cooperation Project (E2024210149), and Public Open Project of Automobile Standardization (CATARC-Z-2025-00934).

** Corresponding author.

(IDSs) therefore monitor message streams and infer abnormal behavior from the fields that are normally available in traces: timestamps, arbitration IDs, DLCs, and payload bytes.

A detector that performs well on a closed split does not necessarily transfer to another vehicle or another attack family. Cross-vehicle evaluation changes the message distribution: the set of frequent IDs, their periods, payload ranges, and local traffic context can differ across vehicles. Unknown-attack evaluation changes the perturbation mechanism: the injected ID, payload manipulation, timing footprint, or replay pattern may differ from the attack families observed during training. These two shifts interact in the CT&T Test04 setting, where both the test vehicle and attack family are unseen during training [17]. In such a setting, a detector that mainly memorizes absolute CAN-ID frequencies or raw payload ranges can fail for reasons unrelated to attack behavior.

Raw fixed-window representations introduce an additional failure mode. If a short burst affects only a few frames, a long window can contain many normal messages and only a small amount of attack evidence. A sequence model or window-level statistic may still learn to find such evidence, but the burden of preserving a sparse local event is moved to the downstream classifier. This motivates a representation that makes local deviations visible before a window is formed.

GRAIN-CAN follows this design principle. It maintains a compact history state for each CAN ID and converts each frame into local residual features relative to recent messages with the same ID. The residuals include same-ID timing gaps, payload-change magnitudes, coarse payload statistics, and local ID-behavior features. The frame-level features are then aggregated with a fixed pre-test window configuration and passed to a supervised classifier. The classifier is intentionally simple; the contribution is the representation pipeline that exposes same-ID local behavior before aggregation.

The paper makes two contributions.

1. We design GRAIN-CAN, a lightweight supervised CAN IDS pipeline that represents each frame with same-ID residuals before window aggregation. It maintains per-ID history states and extracts timing, payload-change, payload-statistical, and local ID-behavior residuals from past traffic. This design targets a specific failure mode of raw fixed windows: short attack evidence can be diluted before it reaches the detector.
2. We implement and evaluate GRAIN-CAN with controlled comparisons that separate representation effects from classifier and metric choices. The evaluation compares raw-window and residual features under the same classifier, includes feature and window analyses, reports attack-positive precision, attack recall, attack-F1, FNR, FPR, and confusion counts as primary IDS metrics, and uses AUPR and Recall@FPR only as supplementary score-based operating metrics.

2 Related Work

Rule-, timing-, and statistical CAN IDS. A first line of CAN IDS work exploits regularities in in-vehicle traffic. Song et al. monitor CAN inter-arrival times and show that message injection often disturbs periodicity [4]. Taylor et al. use ID-frequency anomalies, while Blevins et al. provide a time-based CAN IDS benchmark that confirms timing as an important baseline [5, 10]. Cho and Shin use physical-layer clock skew to fingerprint ECUs, and Viden further uses in-vehicle signal behavior to identify attackers [6, 7]. These methods are lightweight and interpretable, but often depend on a monitored statistic, fingerprint, or hand-set threshold. GRAIN-CAN keeps timing, payload, and ID evidence, but converts them into same-ID residual features and lets a supervised detector combine them after aggregation.

Learning-based and historical CAN IDS. Learning-based detectors replace hand-coded rules with learned representations. CANet proposes an unsupervised neural IDS for high-dimensional CAN traffic and processes messages with different IDs as they arrive [8]. CAN-BERT applies a BERT-style model to tokenized CAN traffic, and IDS-DEC combines a spatiotemporal self-coder with deep embedding clustering [12, 24]. Recent designs explore attention, lightweight transformers, and state-space backbones [25, 26, 29, 23]. HistCAN is especially relevant because it learns historical traffic behavior from benign CAN traffic, giving it a stronger unknown-attack motivation than supervised classifiers [22]. GRAIN-CAN is narrower: it is not a benign-only zero-day detector. It studies whether same-ID residuals before aggregation improve a supervised representation compared with raw fixed windows under vehicle and attack shifts.

Signal-level and multimodal frameworks. Several recent frameworks use semantic information beyond raw bytes. CANShield parses signal-level time series and applies multi-scale autoencoder analyzers with an ensemble decision module [11]. X-CANIDS uses a CAN database to translate payloads into human-understandable signals and support explanations [13]. CANival combines a time-interval analyzer with signal-based analysis, and newer signal-relationship work models dependencies among decoded signals [20, 27]. These systems are stronger when DBC-like information is available and can expose semantic attacks that leave message frequency nearly unchanged. GRAIN-CAN makes a lower-information tradeoff: it uses only timestamps, IDs, DLCs, and payload bytes, which improves portability but provides less semantic visibility.

Graph, federated, and deployment-oriented IDS. Graph-based methods model relationships among messages. StatGraph learns multi-view statistical graphs over CAN messages, and DGIDS dynamically builds CAN message graphs, compares normal feature distributions, and reports attacked-ID localization [14, 21]. Federated IDSs address non-IID data sharing across vehicles, while FPGA, deterministic monitoring, and hardware-counter designs study resource use, prevention, or low-level DoS signals [28, 15, 30, 31]. These works target stronger

relational or deployment goals than GRAIN-CAN. Our focus is a lighter software detector that exposes same-ID evidence before fixed-window classification.

Datasets, benchmarks, and metric comparability. Dataset design and metric reporting strongly affect CAN IDS conclusions. ROAD provides a real automotive benchmark, CT&T separates known and unknown vehicle and attack settings, and can-sleuth analyzes limitations and performance impacts of automotive IDS datasets [9, 17, 18]. ROAD comparative studies, dataset guides, and surveys further show that label unit, split design, feature availability, and metric definitions can change the interpretation of results [19, 32, 16, 33]. Following this lesson, we report attack-positive precision, attack recall, attack-F1, false-negative rate, false-positive rate, and confusion counts as primary thresholded metrics, and retain AUPR and Recall@FPR as supplementary score-based operating metrics [34, 35, 38, 36, 37].

3 Problem and Design Goals

3.1 CAN Stream and Cross-Shift Setting

A CAN trace normally exposes a timestamp, an arbitration ID, a DLC field, and payload bytes. Without decoded signal names, IDS methods must infer attacks from observable regularities such as inter-arrival time, ID frequency, payload evolution, and local message context. Cross-shift evaluation is harder than closed-split evaluation because these observable regularities may change when either the vehicle or the attack family changes.

The CT&T benchmark is useful because it separates vehicle shift and attack shift [17]. Test01 evaluates known vehicles and known attacks. Test02 changes the vehicle while keeping the attack family known. Test03 keeps the vehicle known but changes the attack family. Test04 changes both. This decomposition is important because the same detector can behave differently under vehicle shift and attack shift. ROAD, can-sleuth, and recent comparative work similarly emphasize that IDS claims depend on dataset construction, attack realism, metric definitions, and the availability of comparable predictions or scores [9, 18, 19, 16].

3.2 Window-Level Evidence Dilution

Many attacks are local in both time and ID space. A spoofing, injection, or replay artifact may affect only a small set of messages while the rest of the window remains benign. Directly feeding a raw sequence to a window-level classifier leaves the model to discover which local change is relevant and to keep that evidence visible after pooling, attention, or summary operations. This can work when the training and test distributions are close, but it becomes less reliable when the test vehicle changes and the model must distinguish attack-induced perturbations from benign shifts in ID frequencies or payload ranges.

Figure 1 illustrates the design motivation. A long raw window can represent a sparse abnormal burst as a small fraction of otherwise normal traffic. A same-ID

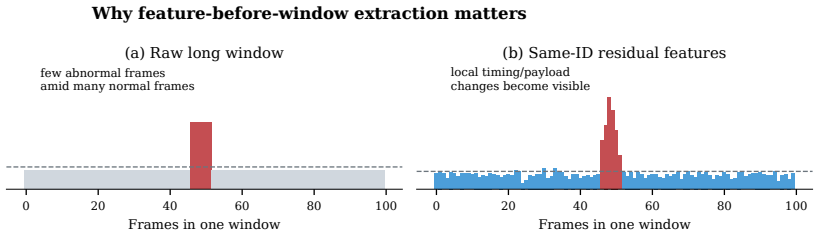

Fig. 1. Feature-before-window extraction exposes sparse local attack evidence before aggregation. A raw long window can dilute a short burst, while same-ID residual features create visible timing or payload-change spikes before the detector sees the window.

residual representation turns the same frames into explicit local changes before window aggregation. The point is not that a residual is vehicle-independent. Instead, residual features shift the detector from absolute values toward changes relative to recent same-ID behavior.

3.3 Design Goal: Residuals Before Aggregation

The central insight is to compute local residuals at the frame level before the window is formed. For a frame with arbitration ID id_i, the detector compares the current timestamp and payload with the recent history of the same ID. Timing residuals capture changes in periodicity or burst rate. Payload residuals capture discontinuities in byte-level evolution. Local ID-behavior features capture short concentration changes. These features are then summarized over a window. The fixed window still provides context, but the local evidence is already explicit when aggregation occurs.

This design is narrower than signal-level or self-supervised frameworks. It does not require a DBC file, decoded signals, or a deep backbone. It also does not claim arbitrary zero-day detection. It is a lightweight supervised representation pipeline for settings in which labeled training data are available and the goal is to improve robustness under vehicle and attack shifts.

4 GRAIN-CAN Design

4.1 Overview

GRAIN-CAN has five stages: input stream parsing, per-ID history-state maintenance, same-ID local behavior feature extraction, fixed pre-test window aggregation, and supervised detection. Figure 2 summarizes the pipeline. The detector operates on the raw fields available in CAN traces and does not assume signal-level decoding. All history-dependent quantities use only current and previous frames.

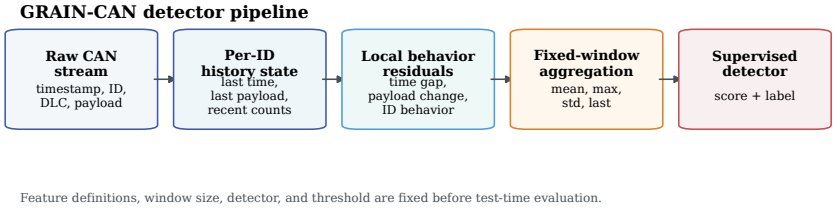

Fig. 2. GRAIN-CAN detector pipeline. The representation is built before the window is formed: raw frames update a per-ID history state, local residuals are extracted from same-ID history, and the resulting frame features are aggregated for supervised classification.

4.2 Input Stream and Per-ID State

The input stream is ordered by timestamp. A frame is denoted as

$$x_i = (t_i, id_i, dlc_i, b_i), \tag{1}$$

where t_i is the timestamp, id_i is the arbitration ID, dlc_i is the data length, and b_i is the payload byte vector. Labels are used for supervised training and evaluation, not for test-time feature extraction.

For each ID, GRAIN-CAN maintains a history state $H(id)$. In the current implementation this state contains the last timestamp, the last payload, recent count statistics, and payload summary values. When a frame arrives, the extractor reads $H(id_i)$, computes residual features, and then updates the state with the current frame. This read-then-update ordering prevents the current frame from being compared with itself.

4.3 Same-ID Residual Feature Construction

The timing residual for frame i is

$$\Delta t_i^{same} = t_i - t_{prev(id_i)}, \tag{2}$$

where $t_{prev(id_i)}$ is the timestamp of the previous frame with the same arbitration ID. If no previous same-ID frame exists, the feature is initialized by a fixed missing-history rule in the feature extractor. This feature captures changes in periodicity, bursts, and replay timing.

The payload-change residual is computed from the current payload and previous same-ID payload. A typical implemented feature is the byte-wise L1 change,

$$\Delta b_i = \sum_{k=1}^{8} |b_{i,k} - b_{prev(id_i),k}|, \tag{3}$$

where $b_{prev(id_i),k}$ is byte k of the previous same-ID payload. Payload summary features include the payload sum, mean, and standard deviation. These summaries

Table 1. GRAIN-CAN pipeline components. The representation modules are fixed before test-time evaluation.

Component	Operation	Control
History state	Stores the last timestamp, last payload, and recent counters for each CAN ID.	Uses past traffic only.
Residual extractor	Computes timing gaps, payload deltas, payload summaries, and local ID behavior.	Reads state before update.
Windowing	Summarizes frame-level residuals into a fixed-length vector.	Window fixed before test.
Detector	Learns a supervised normal/attack boundary and outputs a score or label.	Model and threshold fixed before test.

do not recover signal semantics; they provide coarse byte-level evidence when DBC information is unavailable.

Local ID-behavior features summarize recent ID concentration and frequency. These features are useful for detecting bursts or local changes in bus composition, but they can also encode vehicle-specific information. The experiments therefore distinguish the residual representation used in the main pipeline from feature sets that include raw or overly vehicle-specific terms.

4.4 Window-Level Detector

Frame-level features are aggregated into fixed windows such as $W = 10$, 20, or 100. The window size is fixed before test-time evaluation. A window is labeled attack if any frame in the window is labeled attack. This rule is common in window-level IDS experiments, but it creates normal-dominated attack windows when attacks are sparse. The role of GRAIN-CAN is to make local evidence visible before this label is assigned to the aggregate sample.

The repository experiments instantiate GRAIN-CAN with Gradient Boosting-style classifiers. The classifier is not the methodological novelty. It learns a supervised decision boundary over features produced by the same-ID residual pipeline. When a classifier exposes a continuous score, the paper reports AUPR and Recall@FPR only as supplementary score-based operating evidence. Table 1 summarizes the pipeline components and the leakage controls used in training and evaluation.

4.5 Training and Inference Protocol

Training. Given a fixed window length W, GRAIN-CAN initializes an empty per-ID history table and processes the training stream in timestamp order. For each frame, it reads the current same-ID state, computes timing residuals, payload residuals, payload summaries, and local ID-behavior features, updates the state, and appends the feature vector to the current window. After every W frames, the frame features are aggregated into a window vector. The supervised detector

Table 2. CT&T cross-shift settings used in the main evaluation. Positive rate is computed at the evaluated window or sample unit used by the existing result package.

Setting	Vehicle shift	Attack shift	Frames	Pos. rate	Role
Test01	known	known	16,355,810	0.007	Closed-shift baseline
Test02	unknown	known	17,101,057	0.012	Vehicle-shift stress test
Test03	known	unknown	19,288,415	0.004	Attack-shift stress test
Test04	unknown	unknown	23,873,695	0.003	Joint vehicle/attack-shift stress test

is then trained on window vectors and normal/attack labels. If a score threshold is required, it is selected on validation data or by a pre-declared rule.

Inference. At test time, feature definitions, encoders, window length, detector, and threshold are fixed. The test stream is processed once in timestamp order. Feature extraction uses only current and previous frames. Test labels are used only after inference to compute metrics. The reported W10, W20, and W100 rows are sensitivity configurations, not test-time hyperparameter selection.

5 Experimental Setup

5.1 Dataset and Cross-Shift Splits

The main evaluation uses CT&T set01. Table 2 summarizes the four settings. They form a categorical 2×2 shift decomposition rather than a time series. Test02 isolates vehicle shift, Test03 isolates attack-family shift, and Test04 combines both.

5.2 Compared Pipelines

The main comparison includes representative reproduced pipelines rather than every historical row in the repository. We use short detector names only after their representations and classifiers are defined. *Sample-feature GB* denotes a Gradient Boosting classifier trained on the reproduced public-style sample features. *Sample-feature MLP* uses the same sample-level representation with a multilayer perceptron. *Raw-window Transformer* denotes the existing raw fixed-window neural baseline. *Raw-window GB* summarizes raw W100 windows and trains a Gradient Boosting classifier. *GRAIN-CAN (GB)* uses the same Gradient Boosting classifier family as Raw-window GB, but replaces raw-window statistics with same-ID residual features. Thus, Raw-window GB versus GRAIN-CAN (GB) is the main representation-controlled comparison. Simple timing or payload thresholds are retained only as diagnostic checks and are not treated as recent state-of-the-art competitors.

Table 3. IDS pipelines compared in the evaluation. GB denotes Gradient Boosting. The key controlled comparison is Raw-window GB versus GRAIN-CAN (GB), which uses the same classifier family and W100 window length but different input representations.

Detector	Input representation	Model	Role
Sample-feature GB	Public-style sample features	Grad. Boosting	Reproduced classical baseline
Sample-feature MLP	Public-style sample features	MLP	Neural classical baseline
Raw-window Transformer	Raw W100 frame sequence	Transformer	Raw-window neural baseline
Raw-window GB	Raw W100 window statistics	Grad. Boosting	Same-classifier control
GRAIN-CAN (GB)	Same-ID residual W100 features	Grad. Boosting	Proposed detector pipeline

5.3 Metrics and Operating Points

Attack windows are the positive class throughout the paper. To avoid notation drift between formulas and result tables, we use the following mapping. *Precision* in the tables denotes attack-positive precision P; *Recall* denotes attack recall or detection rate R; and *Attack-F1* denotes F_1^{atk}, the standard F1 score computed with attack as the positive class. Thus Attack-F1 is not a new metric; it is the attack-positive F1 score, used to avoid ambiguity with macro-F1 or weighted-F1:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1^{\mathrm{atk}} = \frac{2PR}{P + R}. \tag{4}$$

where TP is the number of detected attack windows, FP is the number of false alarms, and FN is the number of missed attack windows. We also report false-negative rate (FNR) and false-positive rate (FPR):

$$\mathrm{FNR} = \frac{FN}{TP + FN} = 1 - R, \quad \mathrm{FPR} = \frac{FP}{FP + TN}. \tag{5}$$

where TN is the number of benign windows correctly classified as normal. FNR captures missed attacks, while FPR captures benign traffic falsely reported as attack; both are central to IDS operation [36, 37, 34]. Therefore, the primary thresholded columns in the result tables are *Precision (P)*, *Recall (R)*, *Attack-F1 (F_1^{atk})*, *FNR*, and *FPR*.

For score-producing detectors, the area under the precision-recall curve (AUPR) and recall at a fixed false-positive-rate budget (Recall@FPR) are supplementary operating metrics. AUPR summarizes the precision-recall curve under class imbalance [35, 38]. Recall@FPR fixes a false-alarm budget and reports the best attack recall achievable within that budget:

$$\mathrm{Recall@FPR}(\alpha) = \max_{\tau:\, \mathrm{FPR}(\tau) \le \alpha} \frac{TP(\tau)}{TP(\tau) + FN(\tau)}. \tag{6}$$

where τ is the score threshold and α is the allowed FPR budget. The corresponding missed-attack rate is $\mathrm{FNR@FPR}(\alpha) = 1 - \mathrm{Recall@FPR}(\alpha)$. These score-based metrics can highlight advantages under strict false-alarm constraints, but only among locally reproduced detectors with comparable scores; they are not directly compared with prior papers that report only hard labels or aggregate F1.

5.4 Implementation and Runtime Environment

The repository result package reports W100 as the main fixed-window configuration for the representation-controlled comparison. The same-classifier experiment uses a Gradient Boosting classifier with validation attack-F1 threshold selection. Raw-window GB and GRAIN-CAN (GB) share the same classifier family, train/test split, window length, and metric definitions, so their comparison isolates representation as much as the available artifacts allow. The added efficiency measurements are CPU-only and were repeated three times on the first two CT&T Test04 files; feature extraction time includes CSV parsing and residual construction. We use CPU-only timing because GRAIN-CAN is a streaming feature-extraction and scikit-learn tree-classification pipeline, so GPU acceleration is not part of its proposed deployment path. These measurements quantify current implementation cost, not optimized embedded performance. Classifier sensitivity and rule-threshold diagnostics are treated as supplementary checks rather than main comparisons.

Table 4. Experimental environment used for metric recomputation and CPU-only runtime measurements.

Item	Configuration
OS	Ubuntu 24.04.4 LTS, Linux 6.17.0-35-generic, x86_64
CPU	2× Intel Xeon Gold 5220R, 48 physical cores / 96 threads
Memory	251 GiB RAM, no swap
Software	Python 3.12.3; scikit-learn 1.9.0; NumPy 2.4.4; pandas 3.0.3
Protocol	Runtime measurements are repeated three times; reported values are means.

6 Evaluation

6.1 Cross-Shift Detection

Table 5 summarizes representative CT&T results, including primary thresholded metrics and score-based supplementary columns when comparable scores are available. The main pattern is not that one detector wins every setting. Raw-window Transformer performs strongly in the closed Test01 setting, but its attack-F1 collapses on Test02–Test04 because its high recall comes with excessive false alarms or severe misses. GRAIN-CAN (GB) is less dominant in Test01 but remains substantially more stable under vehicle and attack shifts.

Table 5. Representative CT&T results with attack windows as the positive class. The thresholded primary columns follow Sec. 5.3: Precision (P) is attack-positive precision, Recall (R) is attack recall, Attack-F1 (F_1^{atk}) is attack-positive F1, and FNR/FPR are false-negative/false-positive rates. AUPR and Recall@FPR are supplementary score-based metrics and are reported only when comparable scores are available. Values are rounded to three decimals; "–" denotes unavailable or not recomputed for that row.

Setting	Detector	Precision (P)	Recall (R)	Attack-F1 (F_1^{atk})	FNR	FPR	AUPR	Recall@FPR (10^{-3})
Test01	Sample-feature GB	0.860	0.980	0.916	0.020	0.001	–	–
Test01	Raw-window Transformer	1.000	0.931	0.964	0.069	0.000	–	–
Test01	GRAIN-CAN (GB)	0.999	0.892	0.942	0.108	0.000	0.941	–
Test02	Sample-feature GB	0.931	0.998	0.964	0.002	0.001	–	–
Test02	Raw-window Transformer	0.087	1.000	0.160	0.000	0.133	–	–
Test02	GRAIN-CAN (GB)	0.984	0.931	0.956	0.070	0.000	0.990	–
Test03	Sample-feature GB	0.719	0.629	0.671	0.371	0.001	–	–
Test03	Raw-window Transformer	1.000	0.013	0.025	0.987	0.000	–	–
Test03	GRAIN-CAN (GB)	1.000	0.662	0.797	0.338	0.000	0.925	–
Test04	Sample-feature GB	0.707	0.447	0.548	0.553	0.000	–	–
Test04	Raw-window Transformer	0.011	0.980	0.022	0.020	0.229	0.173	0.146
Test04	GRAIN-CAN (GB)	0.938	0.519	0.668	0.481	0.000	0.785	0.805

Table 6. Representation-controlled comparison using the same Gradient Boosting classifier and W100 windows. Column names follow Sec. 5.3: Precision (P), Recall (R), Attack-F1 (F_1^{atk}), FNR, and FPR are thresholded primary metrics; AUPR and Recall@FPR are supplementary score-based metrics. Values are rounded to three decimals; "–" denotes unavailable or not recomputed for that row.

Setting	Representation	Precision (P)	Recall (R)	Attack-F1 (F_1^{atk})	FNR	FPR	AUPR	Recall@FPR (10^{-3})
Test01	Raw-window statistics	0.999	0.938	0.968	0.062	0.000	–	–
Test01	Same-ID residuals	0.999	0.892	0.942	0.108	0.000	–	–
Test02	Raw-window statistics	1.000	0.895	0.944	0.105	0.000	–	–
Test02	Same-ID residuals	0.985	0.930	0.957	0.070	0.001	–	–
Test03	Raw-window statistics	1.000	0.072	0.134	0.928	0.000	–	–
Test03	Same-ID residuals	1.000	0.661	0.796	0.339	0.000	–	–
Test04	Raw-window statistics	0.694	0.140	0.233	0.860	0.001	0.107	0.140
Test04	Same-ID residuals	0.958	0.633	0.762	0.367	0.000	0.790	0.805

6.2 Representation-Controlled Comparison

The central control experiment compares raw-window statistics and same-ID residual features with the same Gradient Boosting classifier and W100 window length. This isolates representation from classifier choice. Table 6 reports both thresholded metrics and available score-based metrics. Raw-window features are slightly better in Test01, but GRAIN-CAN features improve attack-F1 in all shifted settings. The gain is small in Test02, large in Test03, and large in Test04.

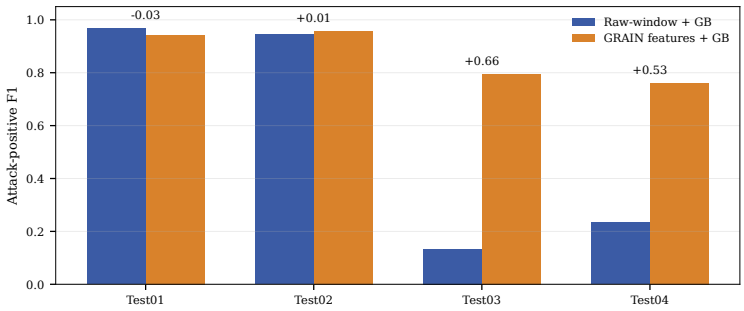

Fig. 3. Representation-controlled comparison. With the same Gradient Boosting classifier, same-ID residual features are not better in the closed Test01 setting, but they improve attack-F1 under the three shifted settings. Numbers above each group denote the attack-F1 change of GRAIN-CAN over raw-window statistics.

Figure 4 visualizes the same result as a shift matrix. This avoids treating Test01–Test04 as a time series. The largest gains occur when attack families are unknown, and the joint-shift Test04 also benefits substantially.

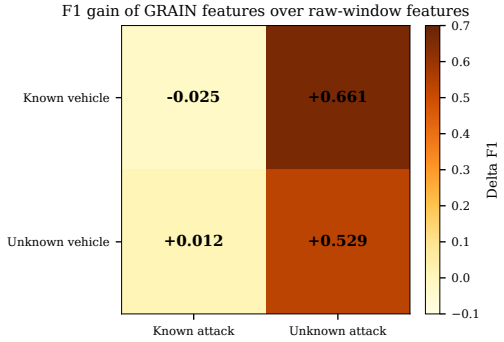

Fig. 4. Attack-F1 gain of same-ID residual features over raw-window statistics in the CT&T vehicle-shift by attack-shift matrix. The categorical matrix is more appropriate than a line plot because Test01–Test04 are not temporal samples.

6.3 Feature Group Analysis

The feature ablation indicates that the main signal in the current CT&T setup is timing residual behavior. On Test04, timing-only features reach attack-F1 0.673, while the full feature set that includes vehicle-specific raw ID and payload terms is weaker under the ablation protocol. Removing CAN-ID behavior improves over the full feature set in the joint-shift setting. This result narrows the method claim: GRAIN-CAN should be viewed primarily as a residual representation pipeline, and the most reliable component in the current evidence is same-ID timing. Payload and ID features may help in some settings, but they should not be overclaimed without additional signal-level evidence.

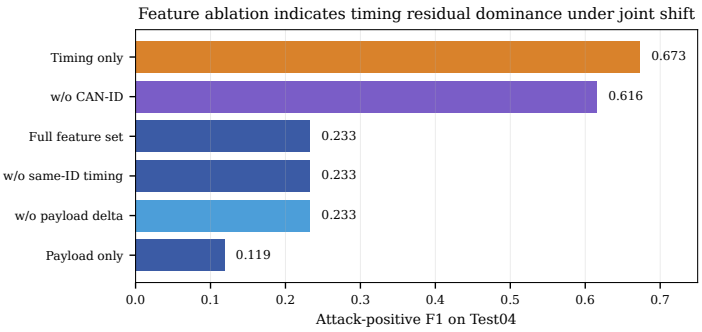

Fig. 5. Feature ablation on Test04. Same-ID timing residuals are the dominant local-behavior signal under joint shift, while raw ID and payload-heavy variants can be less robust.

6.4 Low-False-Alarm Operating Analysis

Recall@FPR is supplementary but important for IDS deployment because it compares detectors at the same false-alarm budget. On Test04, GRAIN-CAN (GB) has AUPR 0.790 and AUROC 0.904, while Raw-window GB has AUPR 0.107 and AUROC 0.570. At an FPR budget of 10^{-3}, GRAIN-CAN achieves Recall@FPR=10^{-3} of 80.5% in the diagnostic operating curve, compared with 14.0% for Raw-window GB. Figure 6 reports several FPR budgets. These values should be read as score-based operating evidence, not as direct comparisons with prior papers that do not expose scores.

Fig. 6. Supplementary low-FPR operating points on Test04. Recall@FPR highlights that same-ID residual features preserve substantially higher attack detection rates than raw-window statistics under matched false-alarm budgets.

6.5 Deployment Cost

The CPU-only measurement shows that the current implementation processes about 97k frames per second end-to-end for GRAIN-CAN (GB) on the measured

Table 7. CPU-only deployment-cost measurement on the first two CT&T Test04 files. The measurement includes stream parsing and feature construction and is repeated three times.

Pipeline	Frames/s	Windows/s	End-to-end ms	Model MB
Raw-window GB	98,269	983	24,497	0.025
GRAIN-CAN (GB)	97,000	970	24,818	0.025

Test04 files. Raw-window GB reaches about 98k frames per second. The difference is small because both pipelines are dominated by CSV parsing and feature construction rather than classifier inference. The model size is about 0.025 MB for both Gradient Boosting variants. This supports a lightweight interpretation of GRAIN-CAN, but it does not substitute for optimized embedded measurements.

6.6 Classifier Sensitivity

A supplementary classifier check fixes the GRAIN-W100 representation and varies the classifier. The residual representation remains effective with logistic regression and random forest variants, indicating that the main evidence is not tied to one Gradient Boosting implementation. Detailed rows are kept in the experiment package rather than the main text.

7 Discussion

7.1 Supported Claims

The strongest supported claim is representation-level. Same-ID residual features before aggregation improve attack-F1 over raw-window statistics under the same classifier in Test02–Test04. The data also show that raw-window neural detection is not uniformly weak: it is strong in Test01 but unstable when vehicle or attack shifts appear. This pattern supports the paper's design motivation without overstating universal superiority.

The ablation results refine the mechanism. Timing residuals are the dominant signal in the current CT&T setup. This suggests that the present performance comes mainly from exposing timing disruptions before window aggregation. Payload and ID features should be treated as modular evidence sources rather than guaranteed contributors under every shift.

7.2 Failure Modes and Boundaries

GRAIN-CAN is supervised. It requires normal/attack labels for training and does not learn a benign-only model. It may miss attacks that preserve same-ID timing, payload continuity, and local ID behavior. It also reduces, but does not remove, dependence on vehicle-specific distributions. Raw ID and payload features can even hurt under joint shift if they encode vehicle-specific patterns. Future versions should consider train-only normalization, signal-level payload features, benign-only pretraining, and online adaptation.

7.3 Comparability to Prior Work

Prior reported aggregate F1 is not directly comparable with attack-F1 unless the positive class, averaging rule, threshold rule, and confusion matrix are known. Similarly, AUPR and Recall@FPR require score outputs and a common score-to-threshold protocol. This paper therefore compares locally reproduced rows under common definitions and treats AUPR and Recall@FPR as supplementary score-based evidence. The paper does not claim that earlier CT&T or ROAD results are wrong; it only separates what can be recomputed from what is externally reported.

8 Conclusion

GRAIN-CAN is a supervised feature pipeline for CAN IDS that makes same-ID local behavior explicit before window aggregation. The representation-controlled experiments show that this design improves over raw-window statistics under vehicle and attack shifts, especially in Test03 and Test04. The added deployment measurement indicates that the current implementation is lightweight on CPU, while ablations show that timing residuals carry the strongest signal. The method remains limited by supervised training assumptions and by attacks that preserve timing and payload continuity. Future work should combine same-ID residual features with self-supervised training, signal-level features, DBC-aware variants, and real-time in-vehicle deployment studies.

References

1. Koscher, K., et al.: Experimental security analysis of a modern automobile. In: IEEE Symposium on Security and Privacy, pp. 447–462 (2010)
2. Checkoway, S., et al.: Comprehensive experimental analyses of automotive attack surfaces. In: USENIX Security Symposium (2011)
3. Miller, C., Valasek, C.: Remote exploitation of an unaltered passenger vehicle. Black Hat USA (2015)
4. Song, H.M., Kim, H.R., Kim, H.K.: Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network. In: ICOIN, pp. 63–68 (2016)
5. Taylor, A., Leblanc, S., Japkowicz, N.: Frequency-based anomaly detection for the automotive CAN bus. In: WCICGSS, pp. 45–49 (2015)
6. Cho, K.T., Shin, K.G.: Fingerprinting electronic control units for vehicle intrusion detection. In: USENIX Security Symposium, pp. 911–927 (2016)
7. Hanselmann, M., Strauss, T., Dormann, K., Ulmer, H.: CANet: An unsupervised intrusion detection system for high dimensional CAN bus data. arXiv:1906.02492 (2019)
8. Verma, M.E., Bridges, R.A., Hollifield, S.C., Iannacone, M.D., Moriano, P.: ROAD: The real ORNL automotive dynamometer controller area network intrusion detection dataset. arXiv:2012.14600 (2020)
9. Bozdal, M., Samie, M., Aslam, S., Jennions, I.: Evaluation of CAN bus security challenges. Sensors 20(8), 2364 (2020)
10. Dupont, G., Lekidis, A., Den Hartog, J., Etalle, S.: Automotive controller area network (CAN) intrusion detection systems on the RAD dataset. arXiv:2408.17235 (2024)
11. Kang, H., Vo, T., Kim, H.K., Hong, J.B.: CANival: A multimodal approach to intrusion detection on the vehicle CAN bus. Vehicular Communications 49, 100845 (2024)
12. Song, J., et al.: DGIDS: Dynamic graph-based intrusion detection system for CAN. Computers & Security 145, 104076 (2024)
13. Zhao, S., et al.: HistCAN: A real-time CAN IDS with enhanced historical traffic learning capability. In: NDSS VehicleSec (2024)
14. Liu, Q., Song, R., Cui, L., Zhang, H., Sun, Y., Sun, L.: MIDS: Detecting stealthy masquerade and tampering attacks on CAN bus via bidirectional Mamba. arXiv:2606.18599 (2026)
15. Shi, J., et al.: IDS-DEC: A novel intrusion detection for CAN bus traffic based on spatiotemporal self-coder and deep embedding clustering. Vehicular Communications 48, 100825 (2024)
16. Rai, R., et al.: Securing the CAN bus using deep learning for intrusion detection in vehicles. Scientific Reports 15, 98433 (2025)
17. Saravanan, R., et al.: Optimal attention deep learning based in-vehicle intrusion detection and classification. Scientific Reports 15, 10637 (2025)
18. Gao, F., et al.: Signal-relationship-aware explainable intrusion detection for in-vehicle networks based on graph transformers. Knowledge-Based Systems (2025)
19. Althunayyan, M., Javed, A., Rana, O.: A robust multi-stage intrusion detection system for in-vehicle network security using hierarchical federated learning. arXiv:2408.08433 (2024)
20. Sharmin, S., Mansor, H., Abdul Kadir, A.F., Aziz, N.A.: Benchmarking frameworks and comparative studies of controller area network intrusion detection systems: A review. arXiv:2402.06904 (2024)
21. Lampe, B., Meng, W.: can-train-and-test: A curated CAN dataset for automotive intrusion detection. Computers & Security 140, 103777 (2024)
22. Kidmose, E., et al.: can-sleuth: Sleuthing out capabilities, limitations, and performance impacts of automotive intrusion detection datasets. In: EICC (2024)
23. Guerra, L., et al.: Automotive intrusion detection systems on the ROAD dataset. arXiv:2408.17235 (2024)

29. Narang, P., et al.: FedLiTeCAN: A federated lightweight transformer for fast intrusion detection on CAN bus. arXiv:2512.24088 (2025)
30. Rotaeche, D., et al.: CANTXSec: Deterministic CAN bus monitoring and prevention. arXiv:2505.00384 (2025)
31. Oberti, F., Di Carlo, S., Savino, A.: CANDoSA: Hardware-counter IDS for CAN DoS attacks. arXiv:2507.14739 (2025)
32. Verma, M.E., et al.: A comprehensive guide to CAN IDS data and the ROAD dataset. PLOS ONE (2024)
33. Altunhayyan, M., Javed, A., Rana, O.: Survey of learning-based IDS for in-vehicle CAN networks. Computer Networks (2026)
34. Fawcett, T.: An introduction to ROC analysis. Pattern Recognition Letters 27(8), 861–874 (2006)
35. Davis, J., Goadrich, M.: The relationship between Precision-Recall and ROC curves. In: ICML, pp. 233–240 (2006)
36. Ulvila, J.W., Gaffney, J.E.: Evaluation of intrusion detection systems. J. Res. NIST 108(6), 453–473 (2003)
37. Gu, G., Fogla, P., Dagon, D., Lee, W., Skoric, B.: Measuring intrusion detection capability: An information-theoretic approach. In: ASIACCS, pp. 90–101 (2006)
38. Saito, T., Rehmsmeier, M.: Precision-recall plots for imbalanced binary classifiers. PLOS ONE 10(3), e0118432 (2015)